

1. When explaining types, many authors compare them to mathematical sets. Explain why this comparison is appropriate and useful.

Types can be compared to mathematical sets because a type represents a “collection” of potential values. For example, the Boolean type is thought of as a set containing true and false.

2. What is the relationship between TypeScript and JavaScript? What was the rationale behind the development of TypeScript? What is the primary feature set that TypeScript adds to JavaScript?

TypeScript is based on JavaScript. It’s basically JavaScript with additional features. TS was made to make it easier to make and control larger JS programs by catching certain mistakes before the program runs. Its main addition is a static type system.

3. The document “TypeScript for the New Programmer” at <https://www.typescriptlang.org/docs/handbook/typescript-from-scratch.html> summarizes the differences between JavaScript and TypeScript by saying, “TypeScript is JavaScript’s runtime with a compile-time type checker.” Elaborate on this statement, explaining it in your own words.

This statement means TypeScript programs run using JavaScript’s runtime behavior, but TS checks the program’s types before execution. For example, TS can detect that you are trying to assign a string to a variable that should contain a number.

4. Does the typing information that TypeScript adds ever impact the way in which the program actually runs? Why or why not?

No. TypeScript’s type annotations are removed when TS code is converted to JS.

5. Explain the difference between primitive types and composite types.

A primitive type represents a basic value, such as a number, string, or Boolean. A composite type is made from and/or contains other values. Objects and arrays are examples.

6. List the data types available in JavaScript, distinguishing between primitive and composite types.

Primitive: string, number, bigint, boolean, undefined, symbol, and null.

Composite types: Object

7. List the data types available in TypeScript.

String, number, bigint, boolean, symbol, undefined, null, object, Array, Tuple, Enum, any, unknown, void, never, Union types, Intersection types, Literal types, Function types, Object types, Type aliases, Interfaces, Generic types

8. Explain the typeof operator in the context of TypeScript versus JavaScript.

For JavaScript, typeof is a runtime operator that returns a string describing the “general type” of a value. TypeScript can use typeof in the same way, but it can also use the result to narrow down a value’s type and perform better type checking.

9. An object can be referred to as a composite type and as a reference type. Explain what is meant by these two terms and what, if any, relationship exists between them.

An object is called a composite type because it can contain multiple values and different types of data. It is also called a reference type because a variable holding an object stores a reference to where that object exists in memory rather than storing the object itself.

10. If you have a variable named myVar that contains a numeric value, can you elsewhere in the same TypeScript program assign that myVar variable a value that is represented in some other datatype, such as a string or a Boolean? Explain your answer.

No. If myVar is declared or inferred as a number in TypeScript, you cannot later assign it an other datatype like a string or Boolean. However, you can allow multiple types by declaring myVar with a union type.

11. How do you compare values to see if they’re equal? Is the method the same for all data types, or are there different methods for different data types? Are there any special considerations or caveats of which you should be aware?

Values can be compared using == or ===, but === is preferred because it compares both the value and type without converting anything. A caveat is objects are different because they are compared by their references, so two separate objects with the same contents are not equal.

12. How would you explicitly convert a string to a number? A number to a string? Anything to a Boolean?

A string can be converted to a number with Number(), a number can be converted to a string with String(), and a value can be converted to a Boolean with Boolean().

13. Which values are considered falsy (that is, are equivalent to a Boolean value of false)? Knowing which values are falsy, what’s the simplest way to define the set of values that are truthy? What’s special about reference types in this regard?

Falsy values are false, 0, -0, 0n, "", null, undefined, and NaN. Everything that is not falsy is considered “truthy”. Objects are truthy even when they look empty.

14. What is a type annotation (aka type assignment)? How are type annotations used to declare and initialize variables?

A type annotation tells TypeScript what type a variable should contain. Example of how it’s written:

```
let grade: number;
```

15. What is a type assertion and how does it differ conceptually from a type annotation? What syntax is used for type assertions?

A type assertion tells TypeScript to treat a value as a particular type when the programmer has information the compiler cannot determine itself. Different from an annotation, it does not declare what type a variable is allowed to contain. Syntaxes:

```
let value: unknown = "snail";  
let text = value as string;  
let text = <string>value;
```

16. What is a type alias (aka custom type) and how are type aliases generally used? What is the applicable syntax?

A type alias creates a custom name for a type. It is useful for making complicated types easier to understand. Syntax:

```
type TypeName = TypeDefinition;
```

17. How does strict mode apply to code executed in the Deno runtime?

Strict checking enables a collection of compiler checks that make TypeScript more cautious about potentially unsafe values and operations. This helps identify problems such as possibly null or undefined values before execution. It's automatically on in Deno.

18. Explain the concept of type inference and why it is useful. To what extent should type inference be embraced (or avoided) when writing TS code? Explain how to view the inferred type of an expression using VS Code.

Type inference is when TypeScript determines the type of a variable based on the value given to it automatically. This is useful because you don't have to always manually specify a type. Type inference should be used when the type is obvious. In VS Code, you can hover your mouse over the variable you want to see the inferred type of.

19. What is the concept of narrowing? What is a type guard and how does it relate to this concept?

Narrowing is when TypeScript takes a variable that could have multiple types and determines which type it is. A type guard is a check in the code that helps TS narrow down the possible type.

20. How might the JavaScript typeof operator be utilized in a TypeScript type guard? What are the limitations of this approach?

The typeof operator can be used as a type guard by checking the type of a value. For example, typeof value === "string" tells TypeScript that the value is a string. A limitation is that typeof cannot identify every type specifically.

21. Describe how a type guard might be constructed based upon the truthiness of a single value. What must one be careful of when utilizing such a type guard?

A truthiness type guard checks whether a value is truthy or falsy, such as by using if (value). If the condition is true, TypeScript knows the value is not one of the falsy values. One must be careful because valid values like 0 and an empty string "" are also considered falsy.

22. Describe how a type guard might be constructed based upon the strict equality of two values.

Strict equality can be used as a type guard by comparing a value using ===. If the comparison is true, TypeScript can narrow the variable to the type or value it was compared against.

23. Describe the behavior of the in operator and explain how it might be used to construct a type guard.

The in operator checks whether a property exists in an object. It can be used as a type guard because TypeScript can use the presence of a certain property to figure out what type of object it is.

24. Describe the behavior of the instanceof operator and explain how it might be used to construct a type guard.

The instanceof operator checks whether an object is an instance of a certain class or constructor. It can be used as a type guard because TypeScript can narrow the object to that specific type when the check is true.

25. Explain the syntax and behavior of TypeScript's non-null assertion operator and describe how it might be used to avoid unnecessary type-checking errors.

The non-null assertion operator is written as ! after a value. It tells TypeScript that the programmer knows the value is not null or undefined. This can prevent unnecessary type-checking errors.

26. What is an enumerated type (aka enumeration or just enum)? What sets them apart from most other features of TypeScript? What is the applicable syntax?

An enumerated type, is a type that represents a group of related named values. It is different from many TypeScript features because enums can create JavaScript code that exists when the program runs. An enum is created using the enum keyword followed by its name and values.

27. What is a union type and how are union types defined?

A union type allows a variable to have more than one possible type. Union types are created by separating the allowed types with |

28. What is an intersection type and how are intersection types defined?

An intersection type combines multiple types into one type. It uses the & symbol.

29. What is a generic and why are generics useful? What is the related syntax?

A generic allows code to work with different types while still keeping TypeScript's type checking. This is useful because the same function or type can be reused with different kinds of data. Generics use < > with a type variable like T.

30. Describe some ways that the clean code philosophy could be meaningfully applied to the material you've discovered in this unit

The clean code philosophy can be applied meaningfully by using clear names, keeping types simple, and avoiding unnecessary code. Type inference can be used when a type is obvious, while type aliases can make complicated types easier to understand. Features like union types and narrowing can also make code easier to read.